BEATUP ENGINE

+ SILENCER

Complete User Manual

Windows x64 • Editor • Config-Driven Patcher • Silencer • BPM Workflow

Purpose of this manual

This document is written as a reading manual, not a screenshot guide. It explains what each component does, how to operate it, how the Editor and Patcher are separated, how Silencer is used for timing preparation, and how to move from raw audio to a tested chart.

Project documentation based on the supplied BeatUp Engine + Silencer guide and the current workflow described for the project. The manual intentionally avoids image placeholders so it can be used as a clean reference document.

GitHub	github.com/bishpop
Website	bishpop.github.io/sanya/
BeatUp Engine	bishpop.github.io/sanya/patcher.html

Table of Contents

Navigation

Each chapter title above is a clickable internal link. Use the browser/navigation pane in Word as an additional way to jump between Heading 1 sections.

1. About the Project

Back to Table of Contents

BeatUp Engine is a native Windows rhythm-game project focused on accurate timing, an authentic BeatUp / Audition-style control layout, chart editing, local patching and creator-oriented tooling. The project is organized around two related applications: BeatUp Engine itself and the separate Silencer utility.

The BeatUp Engine side covers playing charts, editing charts, local content handling and the offline Patcher experience. Silencer is the audio/timing preparation tool used before chart authoring or when a chart needs timing correction.

Component	Primary purpose
BeatUp Engine	Play songs, edit charts, review timing, import/export chart data and provide the main runtime.
Editor	Create and adjust note charts on a beat-based timeline, then review them with AutoPlay.
Patcher	Game-facing offline mode with session-based gameplay modifiers.
Silencer	Inspect audio, calculate and refine BPM/offset information, choose Start MADI and verify timing visually.
BPM Detection Kit	Produce a starting BPM and offset estimate from song audio.

Core principle

BPM, Start MADI and offset should be treated as timing data. Once those values are stable, chart placement becomes much more reliable and long-range drift is much easier to diagnose.

The workflow is intentionally separated: analysis first, alignment second, chart creation third, and final gameplay validation last. This prevents large amounts of chart work being built on incorrect timing assumptions.

2. Installation and File Structure

Back to Table of Contents

The release is designed to be portable. Keep the executable, assets and expected support files together after extracting the release archive. Do not move individual asset folders away from their expected relative locations.

2.1 Basic installation

1. Download the appropriate BeatUp Engine release archive.
2. Extract it into a dedicated folder that you control.
3. Keep the assets directory next to BeatUpEngine.exe.
4. Launch BeatUpEngine.exe.
5. Keep your song, chart and audio library organized so related files can be found consistently.

2.2 Recommended folder structure

```
BeatUp Engine/
├── BeatUpEngine.exe
├── assets/
│   ├── arrows/
│   ├── bars/
│   ├── combo/
│   ├── sounds/
│   └── space/
```

```
├── configuration files (build-dependent)
└── songs / charts / audio
```

Folder	Typical content
assets/arrows	Arrow sprites and gameplay-related arrow visuals.
assets/bars	Lane / bar backgrounds and Patcher mode assets.
assets/space	Spacebar interface elements.
assets/combo	Combo and score digits.
assets/sounds	Gameplay sound effects such as beat and miss sounds.
Songs / charts / audio	Your working library of audio and chart files.

Requirement baseline

The project documentation targets Windows 10 / 11, with 64-bit Windows recommended. Developer builds use CMake and Visual Studio/MSVC with SDL2, SDL2_image, miniaudio and Dear ImGui.

3. Application Modes and Configuration

Back to Table of Contents

A core design goal is to keep the Editor and the Patcher as separate operating experiences. The same application architecture can expose different front-end behavior through configuration instead of mixing editor-only controls into the game-facing Patcher interface.

3.1 Configuration concept

Treat the configuration shipped with your current build as authoritative. Configuration determines which front-end or mode is active and which functionality is exposed. The exact configuration filename may vary between builds.

```
mode = patcher
# or
mode = editor
```

Do not simulate mode changes manually

Do not rename executables or move assets just to force a mode change. Use the configuration mechanism provided by the build so initialization and relative paths remain consistent.

3.2 Editor versus Patcher

Area	Editor	Patcher
Chart creation / editing	Primary purpose	Not the primary purpose
Gameplay review	Yes, including AutoPlay/Play testing	Yes
Patcher modifiers	No	Yes
Chart validation	Yes	Used as part of validation/testing
Configuration separation	Yes	Yes

3.3 Session behavior

Patcher gameplay modifiers are intended to affect the active session rather than become permanent profile data. Under the current design, restarting the application returns the active Patcher modifier to Default.

4. Main Menu and Song Selection

Back to Table of Contents

The main menu is the starting point for selecting songs and entering gameplay or the configured mode. The exact visual arrangement can change between releases, but the basic navigation remains the same.

4.1 Selecting a song

6. Use Up / Down to move through the song list where supported.
7. Press Enter to start the selected song.
8. Keep the related audio and chart resources together so file resolution remains predictable.

4.2 General menu controls

Key	Function
Up / Down	Navigate the song list.
Enter	Start the selected song.
Esc	Go back / return to a previous menu state. In gameplay, the configured long-hold behavior can exit.
Ctrl + S	Open sound settings where supported.

Before playing

If a song opens but its timing looks wrong, do not immediately adjust chart notes. Check BPM, Start MADI and offset in Silencer first.

5. Chart Editor

Back to Table of Contents

The Chart Editor is the content-creation side of the project. It is organized around a timeline canvas, beat-based snapping and live review. The documented grid provides 1/4, 1/8 and 1/16 subdivisions.

5.1 Importing audio and chart content

9. Import the song audio supported by the current build, such as OGG where available.
10. Import the corresponding chart, using the current SLK-based workflow where applicable.
11. Confirm that the audio start position and chart timing are aligned before placing large amounts of notes.

5.2 Timeline snapping

Snap	Recommended use
1/4	Coarse chart placement and fast structural work.
1/8	Detailed rhythmic placement.
1/16	Fine corrections and dense patterns.

5.3 Split view and AutoPlay

Use the split-view workflow to review a chart while keeping the timeline available for editing. AutoPlay is most useful as a timing and pattern verification tool. It should not replace manual listening and visual inspection.

- Check that arrows arrive where the music suggests they should.
- Look for duplicate or conflicting note rows.

- Watch transitions around measure boundaries and special/finish patterns.
- Use later sections of the song to confirm that alignment remains stable.

5.4 Editor controls

Action	Function
Esc	Deselect the current selection or go back, depending on the active editor state.
Delete	Delete the selected content.
Tab	Toggle tab/alignment selection where supported.
Horizontal / vertical editor actions	Navigate and manipulate the current selection according to the active editor state.

6. Gameplay and Controls

Back to Table of Contents

BeatUp Engine uses an Audition / BeatUp-style keypad layout. The right side uses Numpad 9 / 6 / 3 and the left side uses Numpad 7 / 4 / 1. Space and Numpad 5 are used for additional actions or finish moves depending on the chart.

Key	Gameplay meaning
Num 9 / 6 / 3	Right lane: top / middle / bottom.
Num 7 / 4 / 1	Left lane: top / middle / bottom.
Space	Space / special timing input where mapped.
Num 5	Finish-move input where mapped.
Esc	Back / menu / configured long-hold exit.

6.1 Timing windows

The project documentation describes the intended engine timing windows as Perfect ±0.3, Great ±0.4 and Cool ±0.5 in the engine timing model. These are engine-side values; external chart formats may use different units and should not be assumed to share the same convention.

6.2 Mirror and AutoPlay

Mirror changes the visual/input relationship so a pattern can be reviewed from the opposite side. AutoPlay is useful for checking arrow order, holds, finishes and overall pattern timing.

Timing diagnosis

A chart that looks correct at the beginning but slowly moves away from the music later in the song is usually a BPM problem, not an offset problem. Offset sets the starting relationship; BPM determines how the grid progresses over time.

7. Offline Patcher and Game Modes

Back to Table of Contents

The Offline Patcher is the game-facing part of the project. It is deliberately separated from the Editor through configuration so gameplay can be tested without exposing the full chart-authoring workflow.

7.1 Opening the modifier menu

In Patcher mode, use the gear/settings control to access the available gameplay modifiers. The selected modifier is session-scoped. Restarting the application returns to Default under the current design.

7.2 Available modes

Mode	Behavior
Default	Normal, unmodified gameplay.
Rapid	Arrows travel at approximately 2× speed, with the final quarter-beat returning to normal behavior.
Vanishing	Arrows start fully visible and progressively fade toward invisibility as they approach the receptor. The effect is smooth rather than an on/off flash.
Wave	A subtle sine-wave motion is applied to the arrows.
Inverted	Arrow relationships are mirrored: 1 ↔ 9, 2 ↔ 8 and 3 ↔ 7. Input is inverted to match the visual mapping.
Speed Up	Starts at normal 1× speed and progressively increases toward 2×.
Halfway	Arrows begin being drawn from the midpoint of their approach while underlying gameplay timing remains normal.

7.3 Patcher assets

Patcher lane/bar visuals are stored under the bars asset directory. The current named mode assets are:

```
default_c.png
rapid_c.png
vanishing_c.png
wave_c.png
inverted_c.png
speedup_c.png
halfway_c.png
```

8. Patcher AutoPlay and Development Controls

Back to Table of Contents

The Patcher can expose an AutoPlay capability for development/testing without turning that feature into a normal user-facing setting. This is controlled by a deliberately manual configuration key.

8.1 Enabling Patcher AutoPlay

The special key is named dev_mode_autoplay. It is disabled when the key does not exist. The engine does not create this key automatically. To enable it, add the key yourself to the configuration file used by the current build:

```
dev_mode_autoplay=true
```

To disable it explicitly, use:

```
dev_mode_autoplay=false
```

Important configuration rule

The key is intentionally manual. If it is absent, Patcher AutoPlay remains disabled. The normal config-writing path should not invent the key or turn it into a persistent user preference.

8.2 Using Patcher AutoPlay

12. Start the Patcher normally with dev_mode_autoplay enabled in the correct configuration.
13. Enter a song and allow the game to run.
14. Use AutoPlay as a development and verification mechanism: inspect arrow timing, pattern order, mode behavior and general gameplay flow.
15. After changes, repeat the test on more than one chart rather than validating only a single pattern.

8.3 Live modifier hotkeys

During Patcher gameplay, the active modifier can be changed without returning to the settings menu.

Hotkey	Modifier
Ctrl + Alt + 1	Default
Ctrl + Alt + 2	Rapid
Ctrl + Alt + 3	Vanishing
Ctrl + Alt + 4	Wave
Ctrl + Alt + 5	Inverted
Ctrl + Alt + 6	Speed Up
Ctrl + Alt + 7	Halfway

Recommended use

Use live hotkeys for quick A/B testing. Use the gear/settings menu when you want to inspect or deliberately select a modifier before starting a run.

9. Chart Validation and Duplicate Notes

Back to Table of Contents

The Editor includes a live duplicate-note check designed to catch conflicting notes placed on the same beat. When a conflicting row is detected, the row is visually marked red.

9.1 Current duplicate rules

The documented exceptions treat S+F and N+F as valid combinations. Other same-beat conflicts, including N+S, are treated as duplicates for validation purposes.

Combination	Validation
S + F	Allowed exception.
N + F	Allowed exception.
N + S	Duplicate / conflict.
Other conflicting same-beat pairs	Treat as duplicates unless explicitly allowed by the chart rules.

9.2 Validation procedure

16. Scan the chart for red duplicate rows.
17. Resolve all conflicts except documented valid combinations.
18. Use AutoPlay to identify timing mistakes the duplicate checker cannot see.
19. Inspect transitions around measures and special patterns.
20. Re-open the exported chart before final distribution and test it in the Patcher/gameplay runtime.

10. Export and Asset Handling

Back to Table of Contents

10.1 SLK and SSC

The project supports chart workflows involving SLK and SSC. The documented Smart Export concept includes the appropriate audio filename in the exported chart. The SLK export flow may also ask for a credit/watermark string; that value is intended as metadata/credit and does not change the in-game chart behavior.

10.2 Asset replacement

The asset tree is intentionally simple to customize. Keep replacement files in the expected relative directories and preserve filenames whenever the runtime depends on specific names.

- Change one asset at a time when troubleshooting.
- Launch the engine after each change and verify the affected screen still renders.
- Keep backups of original assets before large replacement sets.
- For Patcher mode visuals, check the bars directory and the specific mode asset names.

10.3 Final export check

21. Export the chart.
22. Close or reload the current chart state if necessary.
23. Open the exported file again.
24. Check audio, timing, note order and any metadata/credit field.
25. Perform a final gameplay test in the actual Patcher or gameplay runtime.

11. Silencer Overview

Back to Table of Contents

Silencer is the companion audio/timing utility. Its purpose is to make the difficult timing-preparation step easier to inspect visually by combining waveform viewing, playback, MADI markers, Start MADI selection, offset control and BPM analysis in one workflow.

Function	What it is used for
Waveform view	Visually inspect the complete song and locate strong musical events.
Playback	Listen while watching timing markers move through the song.
Seek	Jump directly to a selected point in the waveform.
Vertical timeline	Inspect the full song while keeping measures visible.
MADI markers	Work with a 4/4 measure reference such as M1, M2, M3.
Start MADI	Choose the measure reference that should define the chart starting point.
Offset	Move the audio lead-in reference without changing the declared BPM.
Volume	Adjust listening level without changing chart timing.
BPM workflow	Use automatic analysis as a starting point and refine manually.

12. Silencer: Loading and Inspecting Audio

Back to Table of Contents

Silencer is intended for common rhythm-game audio sources such as WAV, MP3, OGG and FLAC when the corresponding decoder support is present in the current build.

12.1 Load the song

26. Open the target audio file in Silencer.
27. Wait for the waveform and timing information to become available.
28. Confirm the total duration and inspect the first seconds of the waveform.

12.2 Inspect before changing anything

- Check song duration.
- Check the displayed BPM or analysis state if available.
- Look for silence, a pickup or an intro at the beginning.
- Play from the beginning and listen for the first strong musical beat.

Do not infer offset from silence alone

A song may contain an intro, pickup, breath, FX hit or intentionally weak first beat. Use the waveform, what you hear and the MADI/BPM calculations together.

13. Silencer: Waveform, MADI and Start MADI

Back to Table of Contents

Silencer shows the complete song on a vertically scrollable timeline. In the documented 4/4 workflow, measure markers are displayed as M1, M2, M3 and so on. The waveform is centered so the marker structure can be compared directly with musical events.

13.1 Selecting Start MADI

Click directly on a MADI marker to select Start MADI. The selected value is reflected in the timing display and is then used by the MADI calculation.

29. Find the first musically meaningful measure reference.
30. Move to the exact MADI marker that should act as the chart reference.
31. Click the marker.
32. Confirm that the displayed Start MADI value changes.
33. Continue checking later measures to confirm that the same BPM/offset remains aligned.

13.2 Why Start MADI matters

A chart does not necessarily begin at the first audio sample. Start MADI provides a deliberate measure reference so the charting workflow can distinguish the audio file beginning from the point that should function as the gameplay/chart start reference.

14. Silencer: Offset and Beat Controls

Back to Table of Contents

Offset is treated as a lead-in correction. A positive offset adds lead-in time at the beginning of the audio timeline, while a negative offset removes time from the beginning. The goal is to move the beat reference into alignment without changing the declared musical BPM.

14.1 Beat-step adjustment

The +1 Beat and -1 Beat controls move the offset by exactly one beat duration based on the declared BPM. The current workflow rounds each step to three decimal places.

Action	Effect
+1 Beat	Increase offset by one beat duration.
-1 Beat	Decrease offset by one beat duration.
Positive offset	Adds lead-in time at the start.
Negative offset	Removes time from the beginning.

Example at 240 BPM

One beat is 0.250 seconds. An offset of -0.250 becomes 0.000 after +1 Beat, then 0.250 after another +1 Beat.

14.2 Recommended offset method

34. Estimate the BPM first.
35. Find the first strong musical beat.
36. Select a sensible Start MADI.
37. Apply the smallest offset needed to align the measure grid with the audio.
38. Check a later part of the song.
39. If the alignment drifts, reconsider the BPM instead of continually changing offset.

15. Silencer: Playback, Seeking and Auto-Scroll

Back to Table of Contents

Control	Purpose
Play	Start or resume playback.
Pause	Pause while keeping the current position.
Stop	Stop playback and return to the defined stopped state.
Volume	Adjust listening level without changing timing.
Waveform click	Seek to the selected audio time.

15.1 Auto-scroll

During playback, the timeline follows the current position so the active area remains visible. This allows you to watch measure markers and waveform peaks while listening to the song.

15.2 Practical timing loop

40. Play the current section.
41. Watch where strong waveform peaks occur relative to MADI markers.
42. Pause when the grid visibly or audibly drifts away from the music.
43. Use small offset changes for small corrections; use whole-beat steps only when the mismatch is clearly one beat.
44. Repeat the check later in the song.

Best practice

Always verify timing at more than one point in the song. A correct start can still hide an incorrect BPM that causes gradual drift later.

16. BPM Detection Kit

Back to Table of Contents

The BPM Detection Kit provides a practical starting point for song timing. Its workflow follows the project's ArrowVortex-style analysis approach and evaluates onset/tempo candidates instead of relying on a simplistic first-peak guess.

16.1 Supported audio workflow

- WAV, MP3, OGG and FLAC are supported by the current toolchain when the corresponding decoder is available.
- For command-line workflows, quote file paths that contain spaces.

16.2 Reading the result

Field	Meaning
BPM	Primary tempo estimate, shown to three decimals in the current tool.
Offset	Estimated timing correction; negative values are valid.
Candidates	Alternative tempo candidates returned by analysis.
Format / Size	Basic audio file metadata.
Sample Rate	Audio sampling rate.
Duration	Song length.
Bitrate	Encoded bitrate where available.

16.3 How to use the result correctly

Treat automatic BPM detection as a starting point, not final truth. A wrong-but-plausible BPM can be more damaging than an obviously wrong result because the entire chart may slowly drift out of time.

45. Record the primary BPM and offset.
46. Keep candidate BPM values available if the primary estimate does not line up in Silencer.
47. Load the song into Silencer and compare MADI markers against audible beats.
48. Confirm the selected BPM over a long section of the song before charting heavily.

17. Recommended Chart Production Workflow

Back to Table of Contents

The most reliable production method separates analysis, alignment, charting and final validation. Do not place hundreds of notes before BPM, Start MADI and offset are stable.

Step	Action	Why it matters
1	Prepare the song	Keep the original audio untouched as a backup.
2	Detect BPM	Obtain a primary BPM, offset and useful alternative candidates.
3	Open in Silencer	Inspect duration and the waveform before modifying timing.
4	Choose BPM	Validate the detected BPM over a long section.
5	Set Start MADI	Choose the measure that should be the chart reference.
6	Set offset	Apply the detected value, then refine by listening.

7	Verify long-range timing	A drift later in the song usually means BPM is wrong.
8	Open BeatUp Editor	Use the same timing assumptions as Silencer.
9	Chart with snapping	Start coarse, then move to 1/8 and 1/16 for detail.
10	Run AutoPlay	Verify pattern order, visual timing and gameplay flow.
11	Resolve duplicates	Fix red rows except documented valid combinations.
12	Export and re-open	Verify the actual exported file.
13	Final Patcher test	Test the final chart in the game-facing runtime.

Recommended creator mindset

Silencer solves timing alignment. The Editor solves chart construction. The Patcher solves final gameplay verification. Keep those responsibilities separate and debugging becomes much easier.

18. Troubleshooting and Common Problems

Back to Table of Contents

Problem	Recommended diagnosis
Audio does not load	Confirm the path, verify the format is supported by the current build and make sure required decoder/support files are present.
Chart looks shifted	Re-check Start MADI and offset in Silencer. Do not compensate for an incorrect BPM with a large offset.
Timing drifts later	Re-evaluate BPM. Offset changes the starting position; BPM controls the slope of the timeline.
Patcher mode does not appear	Check the current configuration and confirm the intended Patcher configuration is being loaded from the correct directory.
Custom asset is not visible	Verify the exact relative folder and filename, then restart the application so the asset manager can reload it.
Duplicate warning appears	Inspect the row for two notes on the same beat. S+F and N+F are the documented exceptions.
Seeking or auto-scroll feels wrong	Restart playback and test with a known-good song before diagnosing the chart itself.
Patcher AutoPlay does not activate	Confirm that dev_mode_autoplay=true was manually added to the correct config file. If the key is absent, AutoPlay is disabled.
Live Patcher hotkey does not change mode	Confirm you are in Patcher gameplay and use the exact Ctrl + Alt + number combination assigned to the desired modifier.

Troubleshooting rule

Change one variable at a time. When diagnosing timing, avoid changing BPM, Start MADI and offset simultaneously because you lose the ability to identify which value fixed or caused the problem.

19. Advanced and Release Notes

Back to Table of Contents

19.1 Release builds

Keep distributable release output separate from source and development artifacts. The project targets Windows x64 and uses MSVC/CMake tooling on the developer side.

19.2 Configuration-driven separation

Maintain clear mode boundaries. Initialize only the subsystems needed by the selected mode, keep Editor-only UI out of the Patcher flow and keep gameplay-specific modifier state session-scoped so a restart returns to a predictable baseline.

19.3 Release test checklist

- Test a clean extraction on a second Windows installation or VM.
- Test Editor startup with assets present.
- Test Patcher startup with the intended configuration.
- Test song loading and chart import.
- Test both AutoPlay and manual gameplay.
- Test Silencer with at least one short and one long song.
- Test BPM detection with more than one audio format.
- Test export, re-open and final gameplay after every release candidate.

20. Quick Reference

Back to Table of Contents

20.1 Core workflow

Task	Use
Find BPM	BPM Detection Kit
Refine timing / offset	Silencer
Select Start MADI	Silencer: click a MADI marker
Move offset by one beat	Silencer: +1 Beat / -1 Beat
Create / edit notes	BeatUp Engine Editor
Check chart timing	Editor AutoPlay
Find same-beat conflicts	Editor duplicate-note validation
Test gameplay modifiers	BeatUp Engine Patcher
Change modifier live	Ctrl + Alt + 1 through 7
Enable Patcher AutoPlay	Manual config key: dev_mode_autoplay=true
Replace visuals	assets/ directory
Publish	Export + re-open + final gameplay test

20.2 Core gameplay keys

Key	Use
Esc	Back / menu / exit behavior
Up / Down	Song navigation
Enter	Start selected song
Num 9 / 6 / 3	Right lanes

Num 7 / 4 / 1	Left lanes
Space	Space / special input
Num 5	Finish move where mapped
Ctrl + S	Sound settings where supported
Delete	Delete selection in editor
Tab	Toggle selection/alignment state in editor

20.3 Project links

GitHub: github.com/bishpop

Website: bishpop.github.io/sanya/

BeatUp Engine page: bishpop.github.io/sanya/patcher.html

Manual scope

This manual documents the behavior and workflow supported by the supplied project documentation and the project features discussed for the current build. Exact option names or file locations may vary in future builds; when they do, treat the shipped configuration and UI as authoritative.